

D795

Applied Algorithms and Reasoning

Armored Drill

8 concepts · 12 items · 4 transfer items

What this is. Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

Source. Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	_____	

Items

1.

An algorithm's measured step count is $3n^2 + 500n + 10,000$. It was written in C++ by a senior engineer. What is its Big O?

- A. $O(3n^2 + 500n)$
- B. Better than $O(n^2)$, because C++ is fast.
- C. $O(n)$, because $500n$ is larger for small inputs.
- D. $O(n^2)$

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

2. [TRANSFER — reversed stem / different variable]

Which step-count formula is $O(n \log n)$?

- A. $n^2 / 1000$
- B. $\log n$
- C. $5n \log n + 20n$
- D. $100n$

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

3.

A single-threaded $O(n^2)$ routine takes 2 seconds on 10,000 records. It runs on a laptop with 8 cores. About how long will it take on 40,000 records?

- A. 16 seconds
- B. 32 seconds
- C. 8 seconds
- D. 4 seconds

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

4. [TRANSFER — reversed stem / different variable]

An $O(n^2)$ routine handles 1,000 items in 1 second. About how many items can it handle in 100 seconds?

- A. About 10,000
- B. About 100,000
- C. It can never handle more than 1,000 items.
- D. About 1,000,000

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

5.

A sorted list holds 1,000,000 customer IDs, loaded each morning from a cloud storage bucket. In the worst case, how many comparisons does binary search need to find an ID?

- A. About 6
- B. It depends on how fast the bucket downloads.
- C. About 20
- D. 1,000,000

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

6. [TRANSFER — reversed stem / different variable]

Each binary-search comparison halves the remaining range. About how many sorted items can be searched in at most 12 comparisons?

- A. 24
- B. 144
- C. 1,000,000,000,000
- D. About 4,096

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

7.

An app checks each of 50,000 new usernames against a list of 2 million existing names by scanning the list. The job takes hours. The server has fast SSD storage. What change helps most?

- A. Load existing names into a hash set for average $O(1)$ checks.
- B. Store existing names in a linked list.
- C. Sort the 50,000 new names first.
- D. Upgrade to faster storage.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

8.

A recursive Fibonacci function takes minutes for $n = 50$ because it recomputes the same values again and again. The code is thoroughly commented. What fixes it?

- A. Remove the comments.
- B. Nothing; recursion is always too slow to fix.
- C. Rewrite it in a faster language.
- D. Store each computed value (memoization) so it is calculated once.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

9.

A machine gives change using coins worth 1, 3, and 4 cents. A greedy algorithm (largest coin first) makes 6 cents as $4 + 1 + 1$, or three coins. The machine was installed last spring. What is the fewest coins possible, and what does it show?

- A. It depends on when the machine was installed.
- B. Two coins ($3 + 3$); greedy is not optimal for this coin set.
- C. Six coins of 1 cent.
- D. Three coins; greedy is always optimal.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

10. [TRANSFER — reversed stem / different variable]

Which approach guarantees the fewest coins for any coin set and amount?

- A. Binary search on the coin values.
- B. Dynamic programming over all amounts up to the target.
- C. Greedy, taking the largest coin first.
- D. Greedy, taking the smallest coin first.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

11.

A team spent a week speeding up a function from 50 ms to 10 ms. Later profiling showed each 1,000 ms request spends 900 ms in a database call and runs the function once. The team writes in Go. What was the mistake?

- A. None; the request is now five times faster.
- B. They should speed up more small functions the same way.
- C. They optimized before profiling; the request only dropped from 1,000 to 960 ms.
- D. Go is too slow for this service.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

12.

A dynamic array doubles its capacity when full. Most appends are quick, but occasionally one copies every element. The array stores readings from 40 sensors. What is the amortized cost per append?

- A. $O(1)$
- B. $O(n)$
- C. It depends on the number of sensors.
- D. $O(\log n)$

Target word	The one fact that decides	Answer	☐ sure ☐ unsure ☐ guess
_____	_____	____	

STOP

Answers and dissection follow. Finish every item and every strip first.

Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

1. Answer D Concept 1

An algorithm's measured step count is $3n^2 + 500n + 10,000$. ~~It was written in C++ by a senior engineer.~~ What is its Big O?

Target word: Big O class **Decides:** n^2 grows fastest.

A	Confusion set	Big O does not keep constants or lower-order terms.
B	Camouflage	Language speed changes constants, not the growth class.
C	Adjacent category	Big O describes growth as n gets large, where n^2 wins.
D	Correct	Constants and lower-order terms drop away.

2. Answer C Concept 1 • transfer

Which step-count formula is $O(n \log n)$?

Target word: recognize the class **Decides:** Dominant term is $n \log n$.

A	Confusion set	A small constant does not change the n^2 class.
B	Dropped element	Drops the factor of n .
C	Correct	$n \log n$ outgrows $20n$; the 5 drops away.
D	Adjacent category	That is $O(n)$.

3. Answer B Concept 2

A single-threaded $O(n^2)$ routine takes 2 seconds on 10,000 records. ~~It runs on a laptop with 8 cores.~~ About how long will it take on 40,000 records?

Target word: predicted run time **Decides:** $4\times$ the input means $16\times$ the time.

A	Dropped element	Uses the $16\times$ factor but drops the 2-second base.
B	Correct	$2\text{ s} \times 4^2 = 32\text{ s}$.
C	Confusion set	Scales linearly, as if $O(n)$.
D	Camouflage	Divides by the 8 cores, which a single-threaded routine does not use.

4. Answer A Concept 2 • transfer

An $O(n^2)$ routine handles 1,000 items in 1 second. About how many items can it handle in 100 seconds?

Target word: solve for input size **Decides:** $100\times$ time allows $\sqrt{100} = 10\times$ items.

A	Correct	Time grows with n^2 , so n grows by $\sqrt{100} = 10$.
B	Confusion set	Scales linearly with time.
C	Absolute language	More time allows larger inputs.
D	Inverted operation	Squares the factor instead of taking the root.

5. Answer C Concept 3

A sorted list holds 1,000,000 customer IDs, ~~loaded each morning from a cloud storage bucket~~. In the worst case, how many comparisons does binary search need to find an ID?

Target word: worst-case comparisons **Decides:** $\log_2 1,000,000 \approx 20$.

A	Adjacent category	Uses log base 10 instead of base 2.
B	Camouflage	Loading time does not change the search count.
C	Correct	$2^{20} = 1,048,576 \geq 1,000,000$.
D	Confusion set	Linear search's worst case.

6. Answer D Concept 3 · transfer

Each binary-search comparison halves the remaining range. About how many sorted items can be searched in at most 12 comparisons?

Target word: solve for list size **Decides:** 2^{12} .

A	Confusion set	12×2 treats halving as adding.
B	Inverted operation	12^2 squares instead of raising 2 to the 12th.
C	Adjacent category	10^{12} uses base 10.
D	Correct	$2^{12} = 4,096$.

7. Answer A Concept 4

An app checks each of 50,000 new usernames against a list of 2 million existing names by scanning the list. The job takes hours. ~~The server has fast SSD storage~~. What change helps most?

Target word: data-structure choice **Decides:** Repeated membership checks on a list.

A	Correct	50,000 fast lookups replace 50,000 full scans.
B	Confusion set	Still $O(n)$ per check.
C	Adjacent category	Sorting the wrong list does not speed lookups.
D	Camouflage	The bottleneck is the algorithm, not storage.

8. Answer D Concept 5

A recursive Fibonacci function takes minutes for $n = 50$ because it recomputes the same values again and again. ~~The code is thoroughly commented~~. What fixes it?

Target word: remove repeated work **Decides:** Overlapping subproblems.

A	Camouflage	Comments do not affect run time.
B	Absolute language	Memoized recursion runs fast.
C	Adjacent category	Exponential growth still dominates.
D	Correct	Turns exponential time into linear time.

9. Answer B Concept 6

A machine gives change using coins worth 1, 3, and 4 cents. A greedy algorithm (largest coin first) makes 6 cents as $4 + 1 + 1$, or three coins. ~~The machine was installed last spring.~~ What is the fewest coins possible, and what does it show?

Target word: optimal answer and lesson **Decides:** $3 + 3 = 6$.

A	Camouflage	Install date is irrelevant.
B	Correct	Dynamic programming finds the true minimum.
C	Wrong stem number	The most coins, not the fewest.
D	Absolute language	Greedy fails here.

10. Answer B Concept 6 · transfer

Which approach guarantees the fewest coins for any coin set and amount?

Target word: method with a guarantee **Decides:** Greedy can fail on some sets.

A	Adjacent category	A search method, not an optimizer for combinations.
B	Correct	It checks every combination of subproblems.
C	Confusion set	Fast but not always optimal.
D	Inverted operation	Gives the most coins, not the fewest.

11. Answer C Concept 7

A team spent a week speeding up a function from 50 ms to 10 ms. Later profiling showed each 1,000 ms request spends 900 ms in a database call and runs the function once. ~~The team writes in Go.~~ What was the mistake?

Target word: optimization error **Decides:** The hotspot was the database call.

A	Dropped element	Drops the other 950 ms of the request.
B	Inverted operation	Repeats the mistake on even smaller shares.
C	Correct	The function was 5% of the time; the database was 90%.
D	Camouflage	The language is not the bottleneck.

12. Answer A Concept 8

A dynamic array doubles its capacity when full. Most appends are quick, but occasionally one copies every element. ~~The array stores readings from 40 sensors.~~ What is the amortized cost per append?

Target word: amortized cost **Decides:** Copies are rare and spread out by doubling.

A	Correct	Total copy work is at most about $2n$ over n appends.
B	Confusion set	That is the rare worst case, not the average over many appends.
C	Camouflage	The data source does not change the analysis.
D	Adjacent category	The number of resizes is logarithmic, but the cost per append averages to constant.

Triage grid

Write each item number in the box it belongs to.

	Right	Wrong
Sure	Owned.	Concept inversion — top priority. A wrong mental model; it survives rewording.
Unsure / guess	Recognition, not mastery. Re-drill with the transfer.	Not learned yet.

Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

Trap	What it does	In this drill	Your misses
Dropped element	Leaves out a required term or step.	3	
Wrong stem number	A figure copied from the prompt that is not the answer.	1	
Adjacent category	The right idea in the wrong bucket.	8	
Inverted operation	Direction flipped: add/subtract, cause/effect, higher/lower.	4	
Absolute language	Always, never, guarantee — where judgment applies.	3	
Confusion set	Every option is real; the test is exact role-matching.	9	
Camouflage	Answers a stem fact that was never the question.	8	

Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

#	Concept	Rule	Trap	Tell
1	Big O keeps the dominant term	Big O drops constant factors and lower-order terms and keeps the fastest-growing term.	Keeping constants, or judging by which term is bigger for small inputs.	A step-count formula with several terms.
2	Scaling predictions from growth rate	For $O(n^2)$, multiplying n by k multiplies time by k^2 ; for $O(n)$, by k .	Scaling quadratic work linearly.	A known run time at one size and a question about another size.
3	Binary search	On sorted data, each comparison halves the range, so about $\log_2 n$ comparisons are needed.	Using log base 10 or linear-search counts.	A sorted list and a worst-case comparison count.
4	Hash-based lookup	Hash sets and maps give average $O(1)$ membership checks, versus $O(n)$ for scanning a list.	Swapping one linear structure for another.	Repeated “is it in the list?” checks.
5	Memoization and dynamic programming	Store results of repeated subproblems to trade memory for large time savings.	Blaming recursion or the language instead of repeated work.	A recursive function recomputing the same values.
6	Greedy vs. dynamic programming	Greedy choices are fast but only optimal for problems with the right structure; dynamic programming guarantees the optimum when greedy can fail.	Assuming greedy is always optimal.	A nonstandard coin or weight set.
7	Profile before optimizing	Measure where time is actually spent and optimize the hotspot; speeding up a small share gives a small overall gain.	Optimizing code that feels slow without measuring.	A big local speedup and a small overall change.
8	Amortized analysis	Occasional expensive operations spread over many cheap ones can give a low average; doubling dynamic arrays append in amortized $O(1)$.	Reporting the rare worst case as the per-operation cost.	“Most operations are fast, but occasionally...”

Course scope (public description): This course builds on foundational knowledge of algorithm design and optimization for efficient resource utilization. It emphasizes benchmarking algorithms, reasoning, and applying Big O notation for performance evaluation. This course will cover how to design, analyze, and implement algorithms to solve complex computational problems, focusing on optimizing performance and enhancing system efficiency. The practical applications in this course involve testing data structures, developing algorithmic solutions, and optimizing algorithm performance through critical analysis and implementation. By integrating computational thinking and engineering principles, this course will present the skills students need to develop and optimize algorithms for real-world software, using benchmarking and profiling tools to measure and enhance performance.